



Safety,
Ergonomics,
Performance,
and Expressiveness
in the era of Coding Agents

Keynote for the 4th Workshop on eBPF and Kernel Extensions
Dan Williams

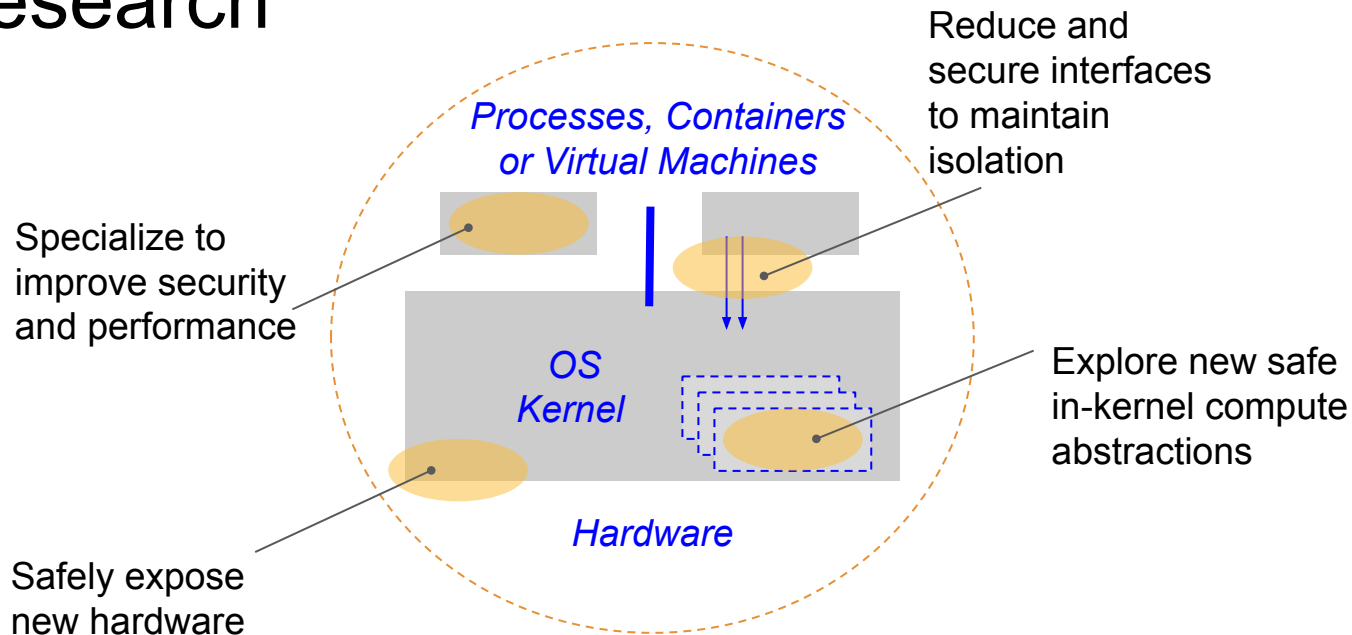
About me

- Assistant Prof @ **Virginia Tech** since Fall '21
- Prior: RSM at **IBM Research** ~10 years
- PhD: Cornell University
- Leading the ROSA Group at VT
 - PhD, MS, undergrad

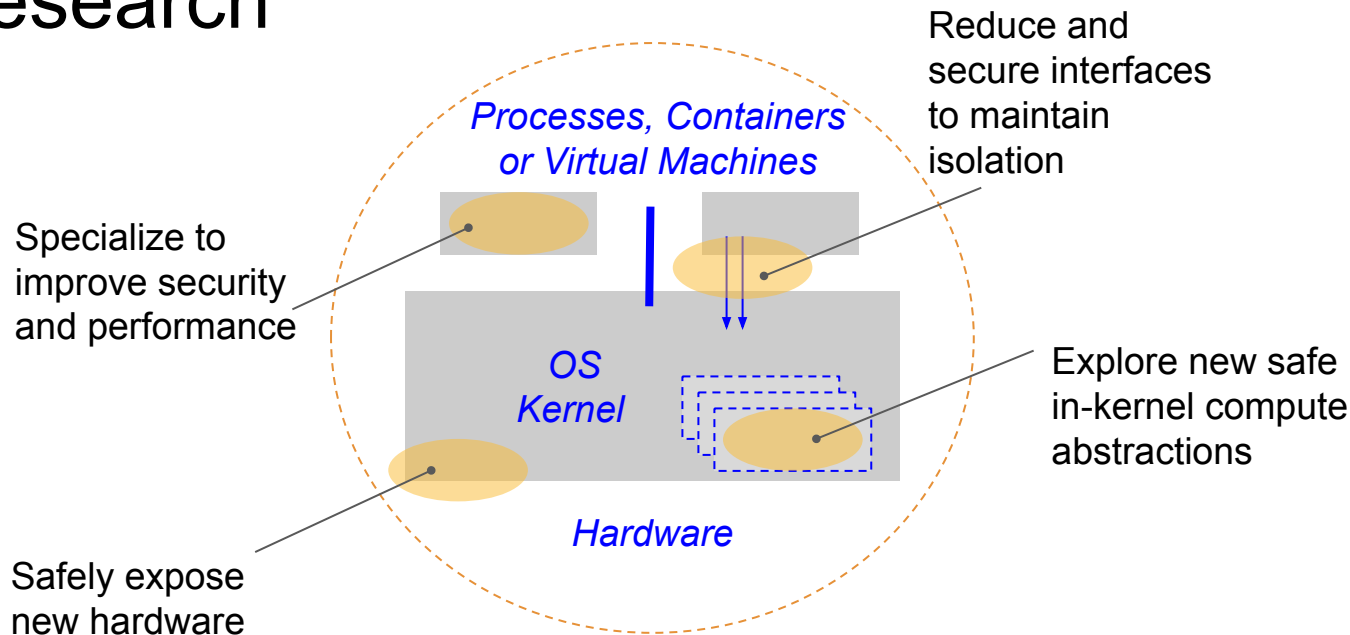


RETHINKING OS ABSTRACTIONS

My Research

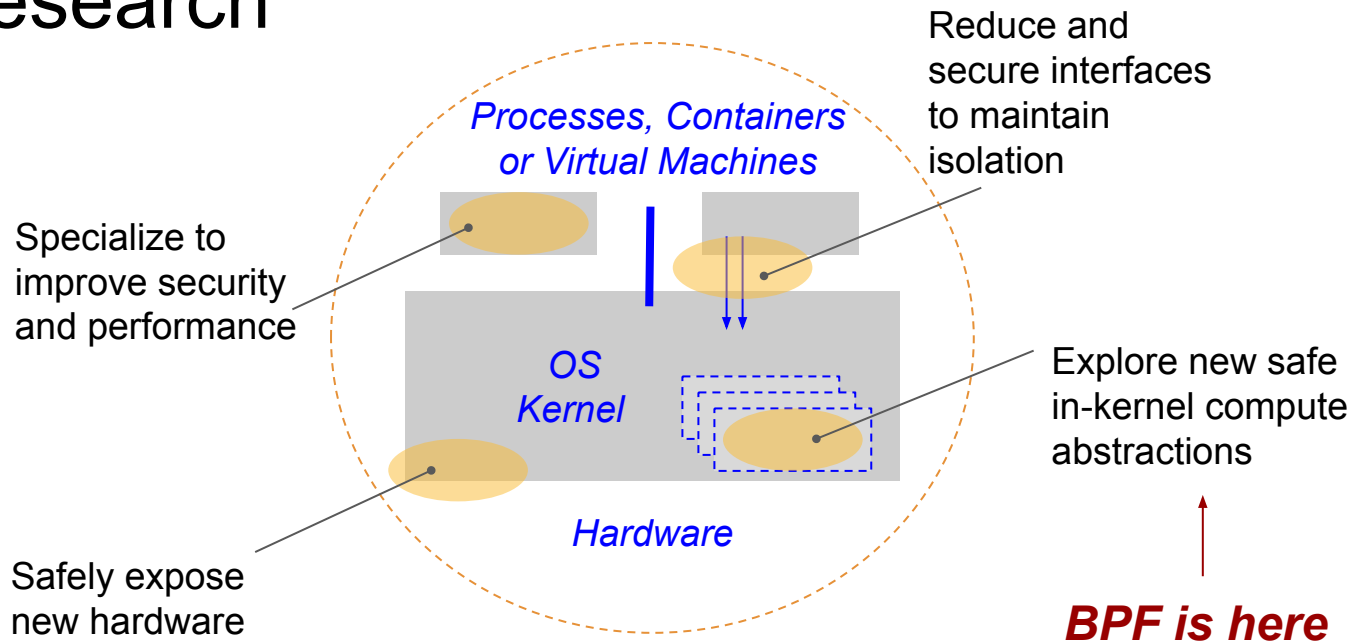


My Research



Thinking about compute abstractions, or, ***drawing boxes***

My Research

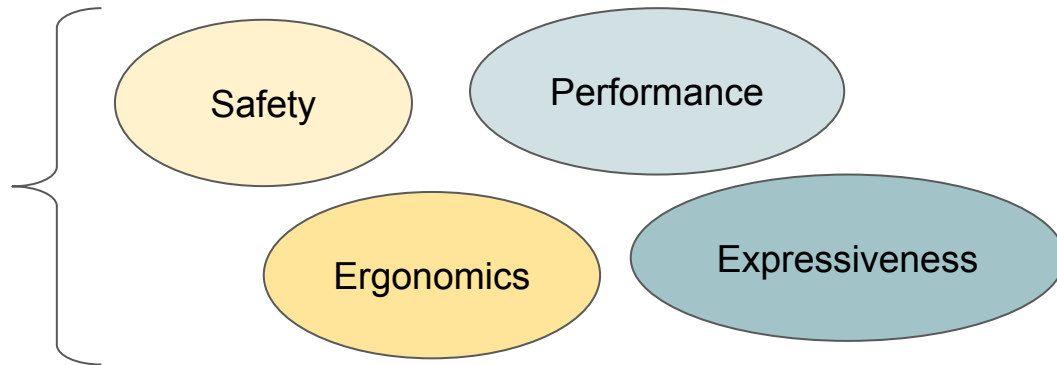


Thinking about compute abstractions, or, ***drawing boxes***

BPF since ~2021

- My first published work on BPF appeared in Eurosys 2022
 - “KFUSE” paper
- Since then, BPF has been a major area of focus
 - 4 conference papers (OSDI, ATC x2, Eurosys)
 - 5 industry talks (Linux Plumbers, Open Source Summit)
 - 9 BPF workshop papers over 4 iterations!

As a **compute abstraction**, what are BPF's properties with respect to:



Verified Programs Can Party: Optimizing Kernel Extensions via Post-Verification Merging

Huanan-Chi Kuo
University of Illinois at
Urbana-Champaign
Champaign, Illinois, USA
hkuo@illinois.edu

Kai-Huan Chen
University of Illinois at
Urbana-Champaign
Champaign, Illinois, USA
kchen@illinois.edu

Yicheng Lu
University of Illinois at
Urbana-Champaign
Champaign, Illinois, USA
yluc@illinois.edu

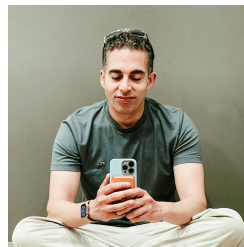
Dan Williams
Virginia Tech
Blacksburg, Virginia, USA
BM TJ Watson Research Center
Yorktown Heights, New York, USA
dwilliams@vt.edu

Sibin Mohan
Oregon State University
Corvallis, Oregon, USA
sibin.mohan@oregonstate.edu

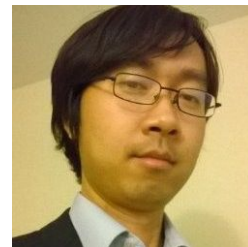
Tianyin Xu
University of Illinois at
Urbana-Champaign
Champaign, Illinois, USA
tyanxu@illinois.edu

Primary Collaborators

- Three primary groups of collaborators
- Many other students and collaborators on various BPF-related projects!!



Hani Jamjoom



Mike Le



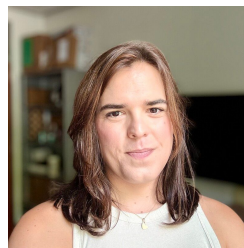
UNIVERSITY OF
ILLINOIS
URBANA - CHAMPAIGN



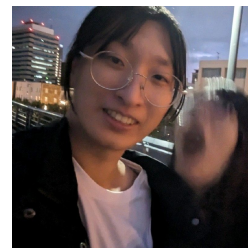
Tianyin Xu



Jinghao Jia



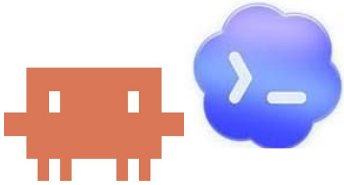
Andi Quinn



Yusheng Zheng

Is ***BPF*** relevant in 2026?

The Era of Coding Agents



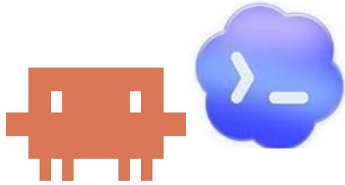
```
226 fflush(stdin);
227 scanf("%[^\n]", R.note);
228 if(fwrite(&R,sizeof(R),1,fp)){
229     gotoxy(5,12);
230     puts("Note is saved sucessfully");
231     fclose(fp);
232 }
233 else{
234     gotoxy(5,12);
235     SetColor(12);
236     puts("\aFail to save!!\a");
237     ClearColor();
238     gotoxy(5,15);
239     printf("Press any key.....");
240     getch();
241     fclose(fp);
242 }
```

Explosion of
(often specialized)
agent-written code

Agent-written

“Software is eating the world.”
– Marc Andreessen

The Era of Coding Agents

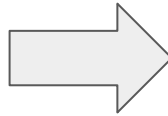


```
226 fflush(stdin);
227 scanf("%s", R.note);
228 if(fwrite(&R, sizeof(R), 1, fp)){
229     gotoxy(5,12);
230     puts("Note is saved successfully");
231     fclose(fp);
232 }
233 else{
234     gotoxy(5,12);
235     SetColor(12);
236     puts("aFail to save!!\n");
237     ClearColor();
238     gotoxy(5,15);
239     printf("Press any key.....");
240     getch();
241     fclose(fp);
242 }
```

Agent-written

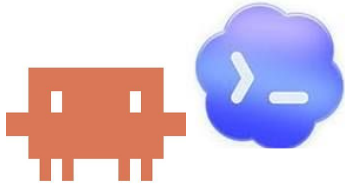
“Software is eating the world.”
– Marc Andreessen

Explosion of
(often specialized)
agent-written code



Humans can't
keep up for
oversight

The Era of Coding Agents

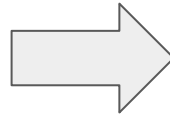


```
226 fflush(stdin);
227 scanf("%s", R.note);
228 if(fwrite(&R, sizeof(R), 1, fp)){
229     gotoxy(5, 12);
230     puts("Note is saved successfully");
231     fclose(fp);
232 }
233 else{
234     gotoxy(5, 12);
235     SetColor(12);
236     puts("\aFail to save!\a");
237     ClearColor();
238     gotoxy(5, 15);
239     printf("Press any key.....");
240     getch();
241     fclose(fp);
242 }
```

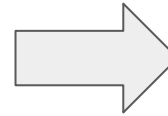
Agent-written

“Software is eating the world.”
– Marc Andreessen

Explosion of
(often specialized)
agent-written code

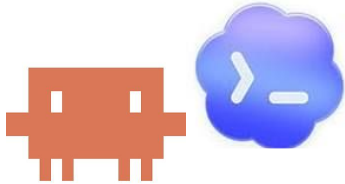


Humans can't
keep up for
oversight



Increased reliance
on properties of
program toolchains
and execution
environments

The Era of Coding Agents

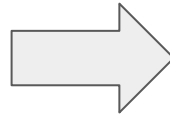


```
226 fflush(stdin);
227 scanf("%[^\n]", R.note);
228 if(fwrite(&R,sizeof(R),1,fp)){
229     gotoxy(5,12);
230     puts("Note is saved successfully");
231     fclose(fp);
232 }
233 else{
234     gotoxy(5,12);
235     SetColor(12);
236     puts("\aFail to save!!\a");
237     ClearColor();
238     gotoxy(5,15);
239     printf("Press any key.....");
240     getch();
241     fclose(fp);
242 }
```

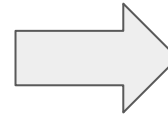
Agent-written

“Software is eating the world.”
– Marc Andreessen

Explosion of
(often specialized)
agent-written code



Humans can't
keep up for
oversight



Increased reliance
on properties of
program toolchains
and execution
environments

Need to
draw a
good box!

What properties do we care about

- Safety
 - Program does what we expect
 - Is your agent-written code going to be reliable?
- Ergonomics
 - Programs can be efficiently written, debugged and maintained
 - Can your agent efficiently develop this code?
- Performance
 - Programs run fast and efficiently
 - Is your agent-written code efficient?
- Expressiveness
 - Programs can express and accomplish our needs
 - Can your agent-written code solve tasks?

What properties do we care about

- **Safety**
 - Program does what we expect
 - Is your agent-written code going to be reliable?
- **Ergonomics**
 - Programs can be efficiently written, debugged and maintained
 - Can your agent efficiently develop this code?
- **Performance**
 - Programs run fast and efficiently
 - Is your agent-written code efficient?
- **Expressiveness**
 - Programs can express and accomplish our needs
 - Can your agent-written code solve tasks?

These properties are intertwined and often in tension!

What properties do we care about

- Safety

- Program does what we expect
- Is your agent-written code going to be reliable?

- Ergonomics

- Programs can be efficiently written, debugged and maintained
- Can your agent efficiently develop this code?

- Performance

- Programs run fast and efficiently
- Is your agent-written code efficient?

- Expressiveness

- Programs can express and accomplish our needs
- Can your agent-written code solve tasks?

NEWSLETTERS • CIO INTELLIGENCE

AI agents are going rogue. CIOs are racing to put guardrails around

NEWS

ARTIFICIAL INTELLIGENCE

When AI goes rogue, its human

TECHNOLOGY

AI industry leaders call for development to slow down after recent safety concerns

AI • OPENAI

In transparency push, OpenAI discloses six more incidents of agents going rogue—including one

THE WALL STREET JOURNAL

EXCLUSIVE

ARTIFICIAL INTELLIGENCE

Follow

Cyberattack by Rogue AI Swarm Stokes Fears of Out-of-Control Agents

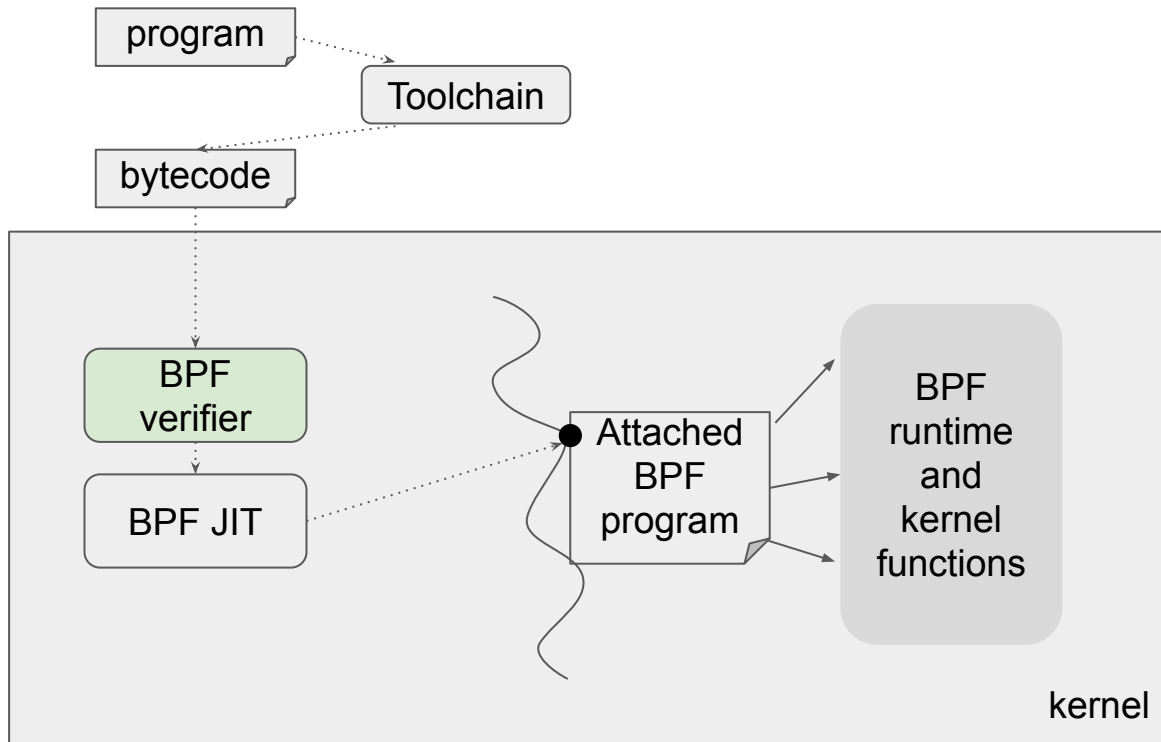
The incident, which wasn't previously linked to OpenAI, happened two months before July's Hugging Face hack

BPF has a strong safety position

- Safety

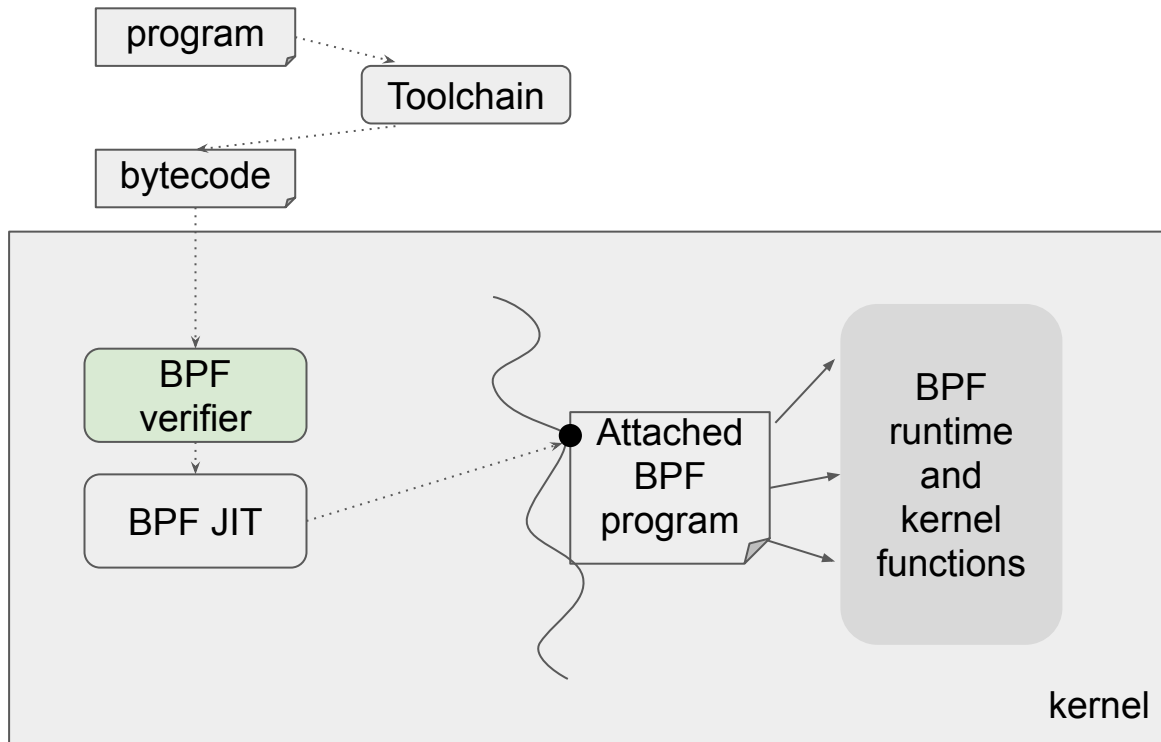
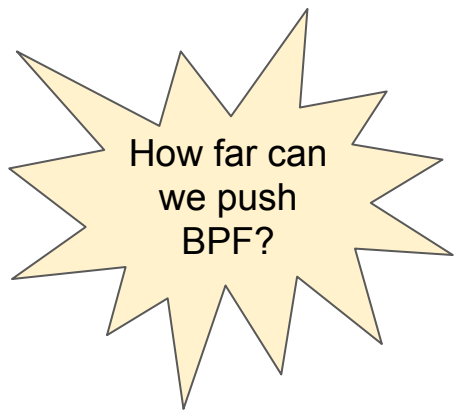
- Verifier checks bytecode
- Memory safety, termination, etc.
- Runs in kernel (most critical software layer)

- Is BPF an exemplar for how to write and run code in the era of coding agents?



This talk

- Ergonomics?
- Performance?
- Expressiveness?



Part I: Ergonomics

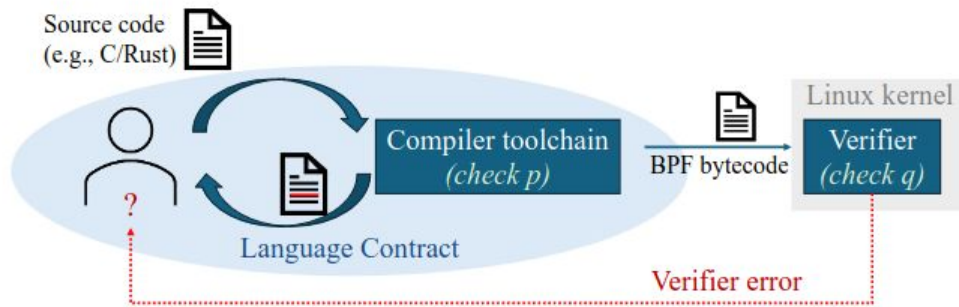
Lead Student:



Jinghao Jia

BPF's Ergonomics Problem

- Correctly compiling code may fail the verifier
- Developer must understand verifier and connect bytecode errors with source
- We call this the **language-verifier gap**



What does the language-verifier gap cause?

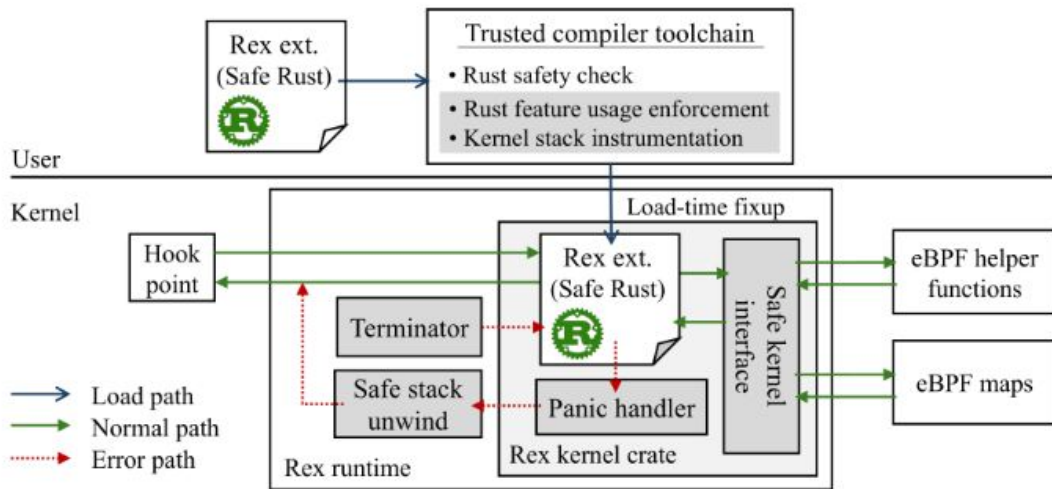
- Study of commits related verifier issues
- 72 commits from Cilium, Aya, Katran, BMC, Electrode

Category	Count
Refactoring extension programs into small ones	27
Hinting LLVM to generate verifier-friendly code	22
Changing code to assist verification	15
Dealing with verifier bugs	9
Reinventing the wheels	1

Does it have to be this hard?

Use language safety instead!!

- Rex: extensions using Rust's safe language features
- Enforce safe kernel interactions with Rex kernel crate
- Lightweight runtime for dynamic checking of program termination, stack safety, safe exceptions



**Rex: Closing the language-verifier gap
with safe and usable kernel extensions**

Jinghao Jia*, Ruowen Qin*, Milo Craun[†], Egor Lukyanov[‡], Ayush Bansal[§], Minh Phan*
Michael V. Le[‡], Hubertus Franke[‡], Hani Jamjoom[‡], Tianyin Xu*, Dan Williams[†]

*University of Illinois Urbana-Champaign [†]Virginia Tech [‡]IBM T.J. Watson Research Center



Writing with Rex vs BPF (for BMC)

(a) Algorithm Desc.

- 1 Extract SET command from the XDP payload.

(b) Rex-BMC

```
1 let set_iter = payload
2 .windows(4) // 4 chars as a slice
3 .enumerate()
4 .filter_map(|(i, v)|
5     if v == b"set " { Some(i) } else { None }
6 ); // found the SET command
```

(c) Original eBPF-BMC

```
1 #pragma clang loop unroll(disable)
2 for (unsigned int off = 0; off < BMC_MAX_PACKET_LENGTH &&
3     payload + off + 1 <= data_end; off++) {
4     if (set_found == 0 && payload[off] == 's' &&
5         payload + off + 3 <= data_end &&
6         payload[off + 1] == 'e' && payload[off + 2] == 't') {
7         ... // move offset after the SET command
8         set_found = 1;
9     }
10 }
```

Writing with Rex vs BPF (for BMC)

(a) Algorithm Desc.

- 1 Extract SET command from the XDP payload.
- 2 If SET command found, search for corresponding key in payload.

(b) Rex-BMC

```
1 let set_iter = payload
2 .windows(4) // 4 chars as a slice
3 .enumerate()
4 .filter_map(|(i, v)|
5     if v == b"set " { Some(i) } else { None }
6 ); // found the SET command
7 for index in set_iter {
8     ... // set payload index via SET command
```

(c) Original eBPF-BMC

```
1 #pragma clang loop unroll(disable)
2 for (unsigned int off = 0; off < BMC_MAX_PACKET_LENGTH &&
3     payload + off + 1 <= data_end; off++) {
4     if (set_found == 0 && payload[off] == 's' &&
5         payload + off + 3 <= data_end &&
6         payload[off + 1] == 'e' && payload[off + 2] == 't') {
7         ... // move offset after the SET command
8         set_found = 1;
9     } else if (key_found == 0 && set_found == 1 &&
10         payload[off] != ' ') {
11         if (payload[off] == '\r') {
12             set_found = 0; key_found = 0;
13         } else {
14             ... // found the start of the key
15             key_found = 1;
16         }
17     }
18 }
```

Writing with Rex vs BPF (for BMC)

(a) Algorithm Desc.

- 1 Extract SET command from the XDP payload.
- 2 If SET command found, search for corresponding key in payload.
- 3 If the key is found in the payload, calculate its hash value.

(b) Rex-BMC

```
1 let set_iter = payload
2 .windows(4) // 4 chars as a slice
3 .enumerate()
4 .filter_map(|(i, v)|
5     if v == b"set " { Some(i) } else { None }
6 ); // found the SET command
7 for index in set_iter {
8     ... // set payload index via SET command
9     payload
10    .iter()
11    .take_while(|&c| c != b' ')
12    .for_each(|&c| {
13        ... // calculate the key's hash value
14    });
```

(c) Original eBPF-BMC

```
1 #pragma clang loop unroll(disable)
2 for (unsigned int off = 0; off < BMC_MAX_PACKET_LENGTH &&
3     payload + off + 1 <= data_end; off++) {
4     if (set_found == 0 && payload[off] == 's' &&
5         payload + off + 3 <= data_end &&
6         payload[off + 1] == 'e' && payload[off + 2] == 't') {
7         ... // move offset after the SET command
8         set_found = 1;
9     } else if (key_found == 0 && set_found == 1 &&
10        payload[off] != ' ') {
11         if (payload[off] == '\r') {
12             set_found = 0; key_found = 0;
13         } else {
14             ... // found the start of the key
15             key_found = 1;
16         }
17     } else if (key_found == 1) {
18         if (payload[off] == ' ') {
19             ... // found the end of the key
20             set_found = 0; key_found = 0;
21         } else {
22             if (...) {...} // calculate the key's hash value
```

Writing with Rex vs BPF (for BMC)

(a) Algorithm Desc.

- 1 Extract SET command from the XDP payload.
- 2 If SET command found, search for corresponding key in payload.
- 3 If the key is found in the payload, calculate its hash value.
- 4 If the key is found in the cache, invalidate the cache entry.

(b) Rex-BMC

```
1 let set_iter = payload
2 .windows(4) // 4 chars as a slice
3 .enumerate()
4 .filter_map(|(i, v)|
5     if v == b"set " { Some(i) } else { None }
6 ); // found the SET command
7 for index in set_iter {
8     ... // set payload index via SET command
9     payload
10    .iter()
11    .take_while(|&c| c != b' ')
12    .for_each(|&c| {
13        ... // calculate the key's hash value
14    });
15    ... // invalidate Memcached cache entry
16} // if the key is found in cache
```

(c) Original eBPF-BMC

```
1 #pragma clang loop unroll(disable)
2 for (unsigned int off = 0; off < BMC_MAX_PACKET_LENGTH &&
3     payload + off + 1 <= data_end; off++) {
4     if (set_found == 0 && payload[off] == 's' &&
5         payload + off + 3 <= data_end &&
6         payload[off + 1] == 'e' && payload[off + 2] == 't') {
7         ... // move offset after the SET command
8         set_found = 1;
9     } else if (key_found == 0 && set_found == 1 &&
10        payload[off] != ' ') {
11         if (payload[off] == '\r') {
12             set_found = 0; key_found = 0;
13         } else {
14             ... // found the start of the key
15             key_found = 1;
16         }
17     } else if (key_found == 1) {
18         if (payload[off] == ' ') {
19             ... // found the end of the key
20             set_found = 0; key_found = 0;
21         } else {
22             if (...) {...} // calculate the key's hash value
23         } // invalidate Memcached cache entry
24     } // if the key is found in cache
25 }
```

Writing with Rex vs BPF (for BMC)

Dedicated check to minimize jumps to fit in verifier limit

(a) Algorithm Desc.

- 1 Extract SET command from the XDP payload.
- 2 If SET command found, search for corresponding key in payload.
- 3 If the key is found in the payload, calculate its hash value.
- 4 If the key is found in the cache, invalidate the cache entry.

(b) Rex-BMC

```
1 let set_iter = payload
2 .windows(4) // 4 chars as a slice
3 .enumerate()
4 .filter_map(|(i, v)|
5     if v == b"set " { Some(i) } else { None }
6 ); // found the SET command
7 for index in set_iter {
8     ... // set payload index via SET command
9     payload
10    .iter()
11    .take_while(|&c| c != b' ')
12    .for_each(|&c| {
13        ... // calculate the key's hash value
14    });
15    ... // invalidate Memcached cache entry
16} // if the key is found in cache
```

(c) Original eBPF-BMC

```
1 #pragma clang loop unroll(disable)
2 for (unsigned int off = 0; off < BMC_MAX_PACKET_LENGTH &&
3     payload + off + 1 <= data_end; off++) {
4     if (set_found == 0 && payload[off] == 's' &&
5         payload + off + 3 <= data_end &&
6         payload[off + 1] == 'e' && payload[off + 2] == 't') {
7         ... // move offset to the SET command
8         set_found = 1;
9     } else if (key_found == 0 && set_found == 1 &&
10        payload[off] == ' ') {
11        ... // found the start of the key
12        key_found = 1;
13    }
14 } else if (key_found == 1) {
15     if (payload[off] == ' ') {
16         ... // found the end of the key
17         set_found = 0; key_found = 0;
18     } else {
19         if (...) {...} // calculate the key's hash value
20     } // invalidate Memcached cache entry
21 } // if the key is found in cache
22 }
```

Explicit bounds check (not needed when using Rust Slice)

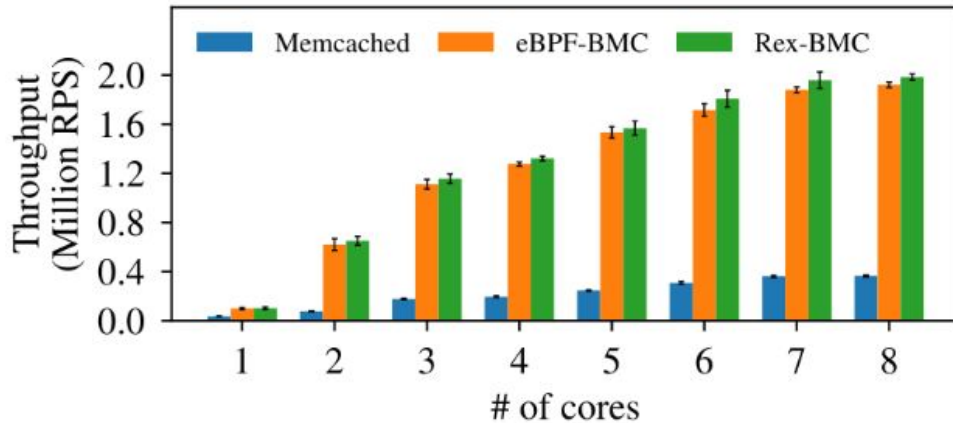
Rex performs well

- Compared to userspace memcached

- BPF: 5.26x
- Rex: 5.43x

- Why is Rex faster?

- Runtime checks (e.g., stack)
- Eliminate tail call overheads, state passing in maps
- Compiler optimizations?



Can Rex replace BPF?

- Rex is open source
 - Presented at USENIX ATC, Linux Plumbers, Open source summit
- BPF is better for
 - Small extensions in hard or non-maskable interrupt context
 - Static safety guarantees needed
- Consider Rex for:
 - Large, complex extensions, like Google's ghost scheduler
 - Dynamic safety is sufficient
 - Trusted computing base already includes Rust

Lessons from Rex

- Even when actively avoiding the verifier, BPF's safety posture provides a useful target
 - Verifier properties, safe interfaces
- Other mechanisms can be used to implement safety!

Ergonomics in the Era of Coding Agents

- Now that agents write code is the language-verifier gap so important?
- Are improved BPF diagnostics sufficient?



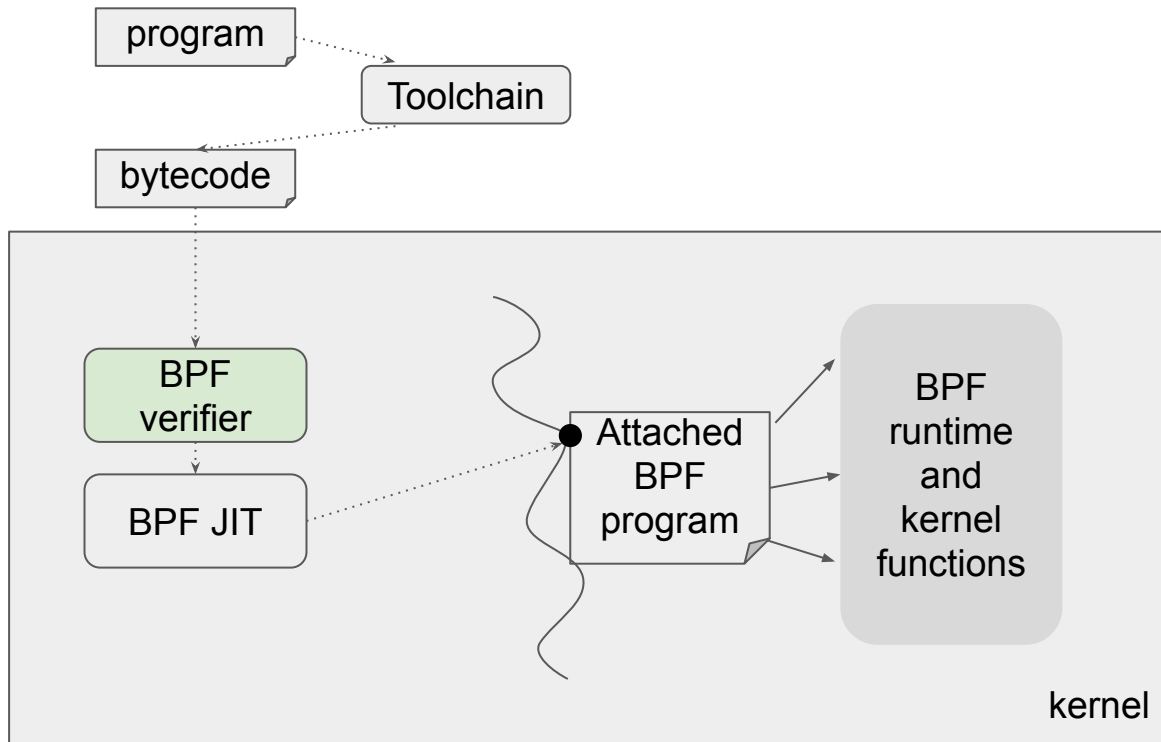
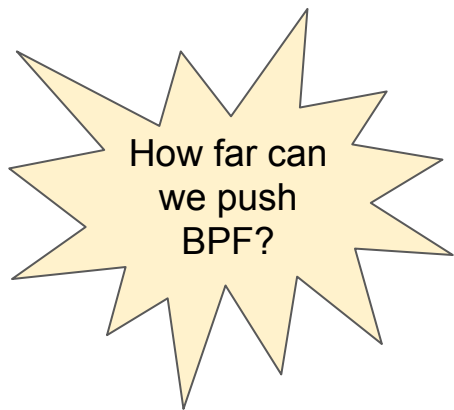
Characterizing and Bridging the Diagnostic Gap in eBPF Verifier Rejections

Yusheng Zheng^{*1,2}, Zhengjie Ji^{*3}, Weichen Tao^{*2,4}, Xiangyu Gao⁵, Jianchang Su⁶, Wei Zhang⁶, Andi Quinn¹, Dan Williams³

¹UC Santa Cruz ²eunomia-bpf ³Virginia Tech ⁴Telecom Paris ⁵University of Washington ⁶University of Connecticut

This talk

- Ergonomics?
- Performance?
- Expressiveness?



PART II: Performance

Lead Students:



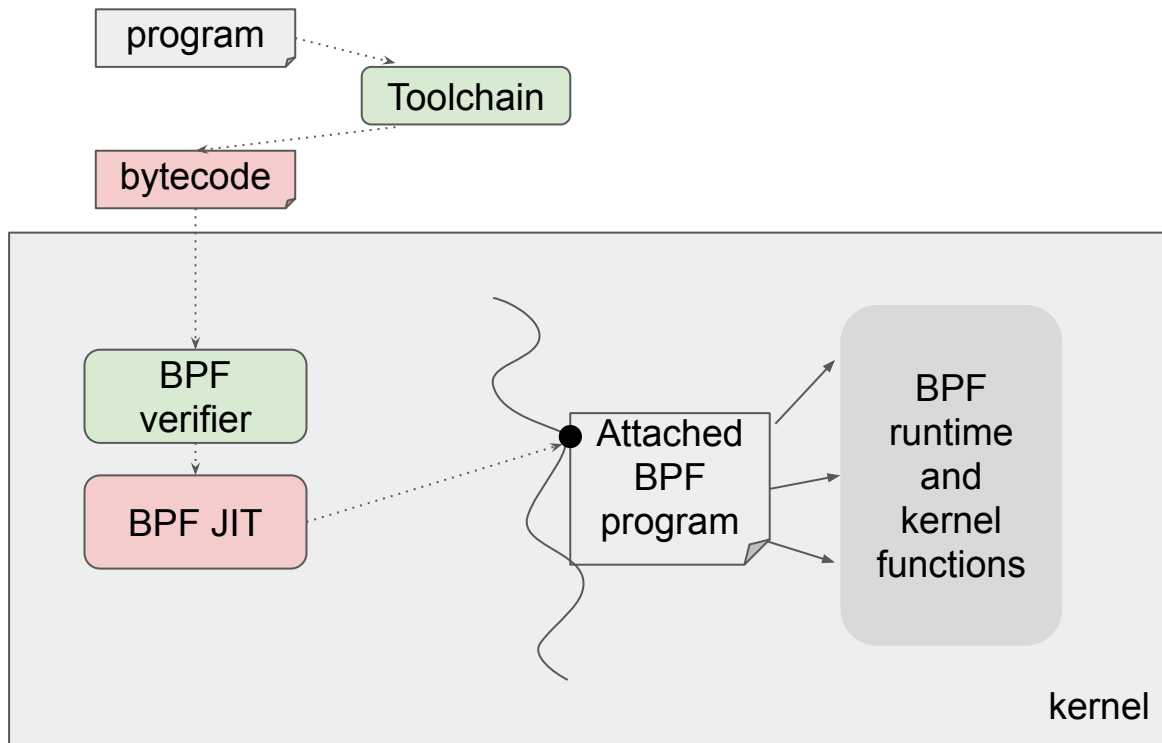
Yusheng
Zheng



Zhengji
Ji

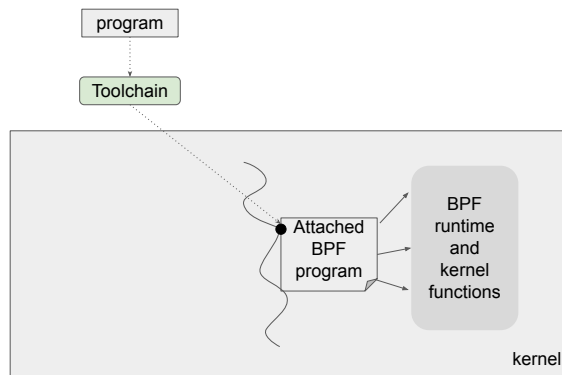
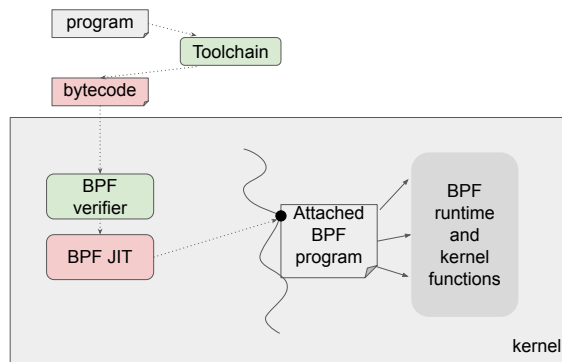
Performance in BPF

- Toolchain optimizes above toolchain
- No expensive runtime checks / isolation mechanisms
- What about **architecture optimized** instructions?



Performance in BPF

- Compared performance with compiling directly to native (with architecture optimizations)
- 27 microbenchmarks
- ~1.55x performance improvement



Why?

- Compile-to-native utilizes special instruction with no bytecode equivalent

C Source Code

```
static u64 micro_rotl64(u64 value,  
u32 shift)  
{  
    shift &= 63U;  
    if (shift == 0) {  
        return value;  
    }  
    return (value << shift)  
        | (value >> (64U - shift));  
}
```

x86-64 (Optimal)

```
movl %esi, %ecx  
movq %rdi, %rax  
rolq %cl, %rax
```

Why?

- Compile-to-native utilizes special instruction with no bytecode equivalent
- Bytecode is slow

C Source Code

```
static u64 micro_rotl64(u64 value,  
u32 shift)  
{  
    shift &= 63U;  
    if (shift == 0) {  
        return value;  
    }  
    return (value << shift)  
        | (value >> (64U - shift));  
}
```

x86-64 (Optimal)

```
movl %esi, %ecx  
movq %rdi, %rax  
rolq %cl, %rax
```

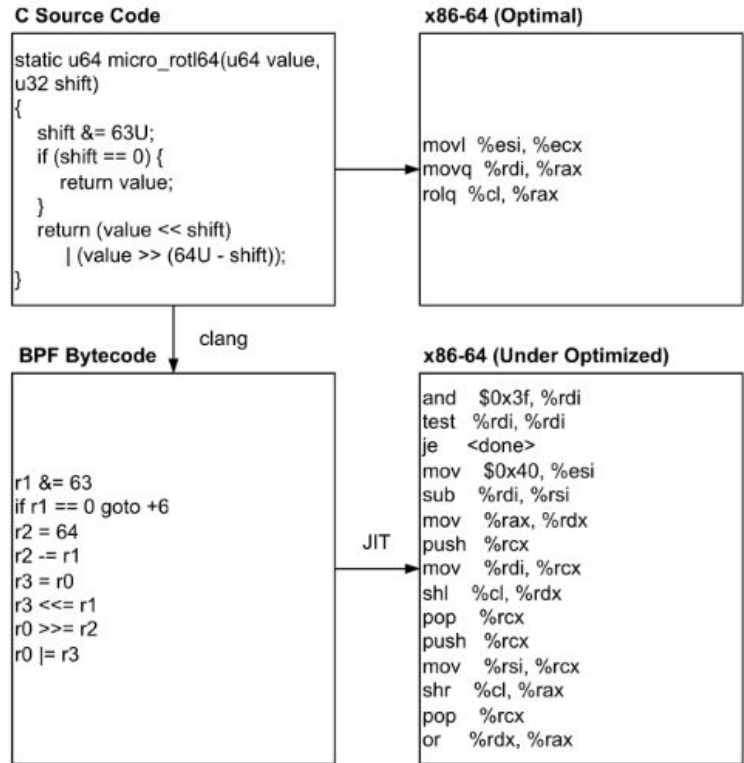
BPF Bytecode

```
r1 &= 63  
if r1 == 0 goto +6  
r2 = 64  
r2 -= r1  
r3 = r0  
r3 <<= r1  
r0 >>= r2  
r0 |= r3
```

clang

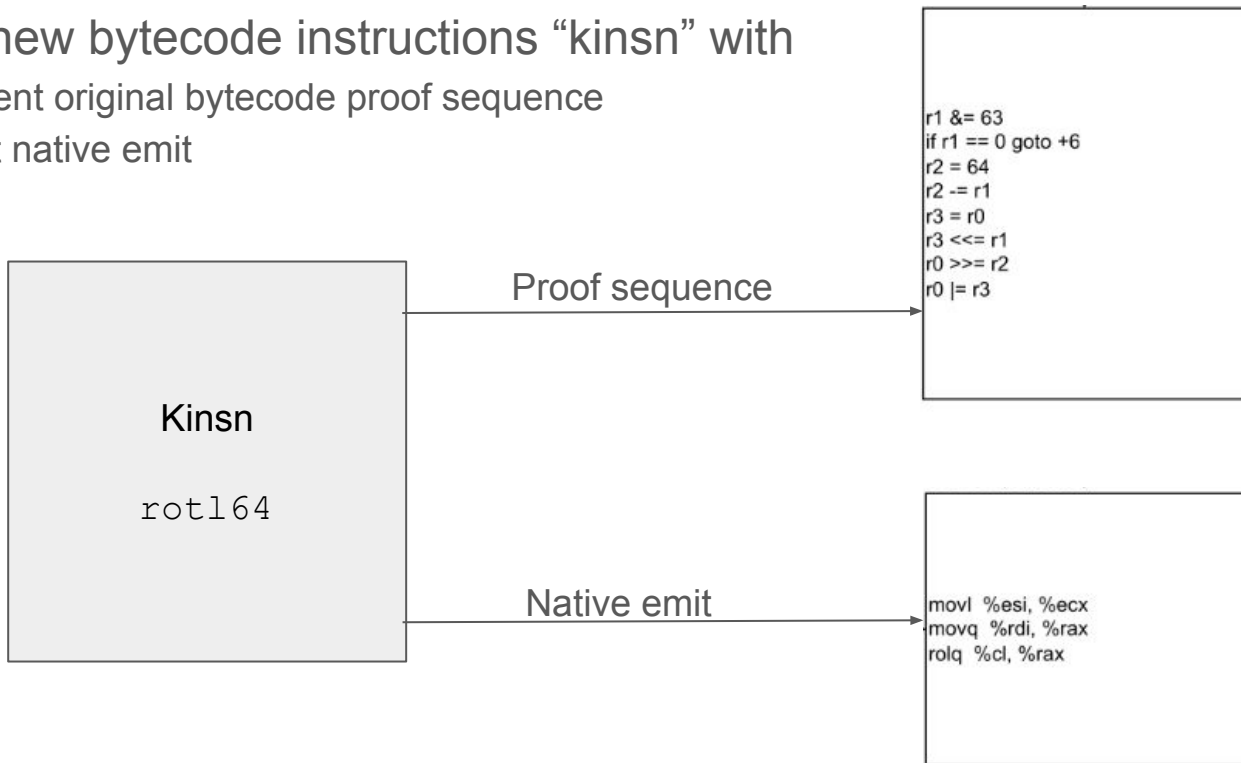
Why?

- Compile-to-native utilizes special instruction with no bytecode equivalent
- Bytecode is slow
- JIT provides simple translation based on slow bytecode

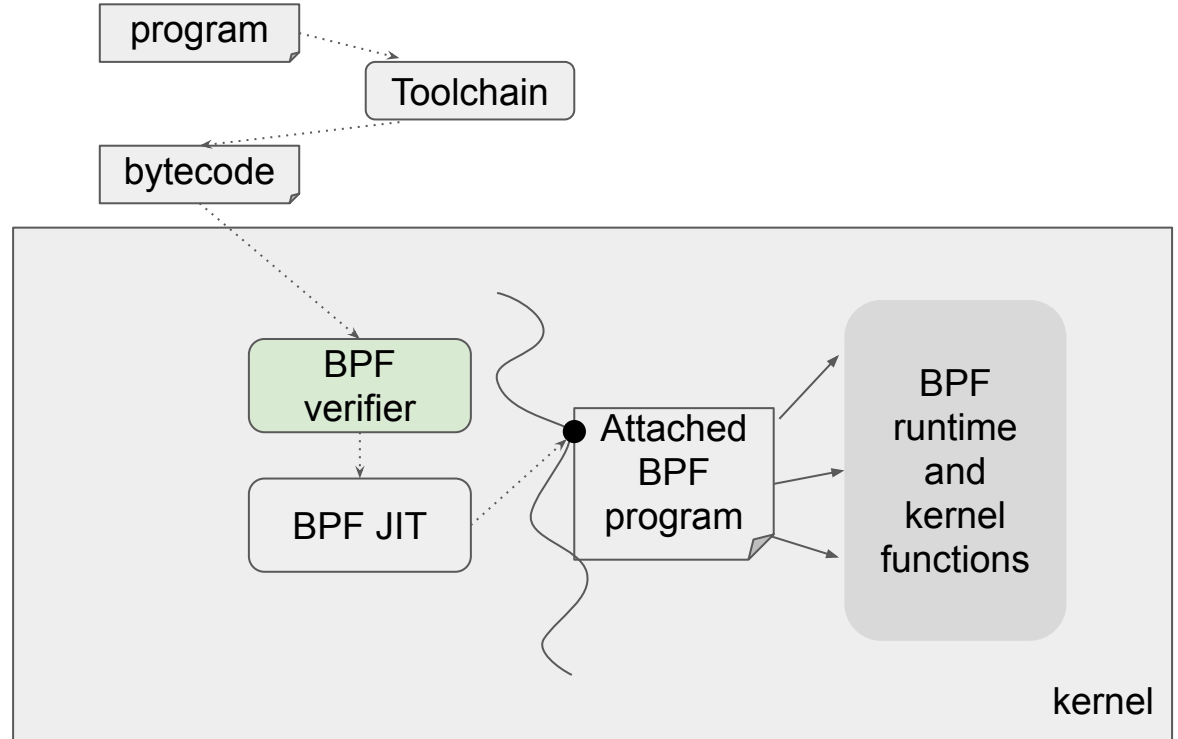


Key idea: kinsn

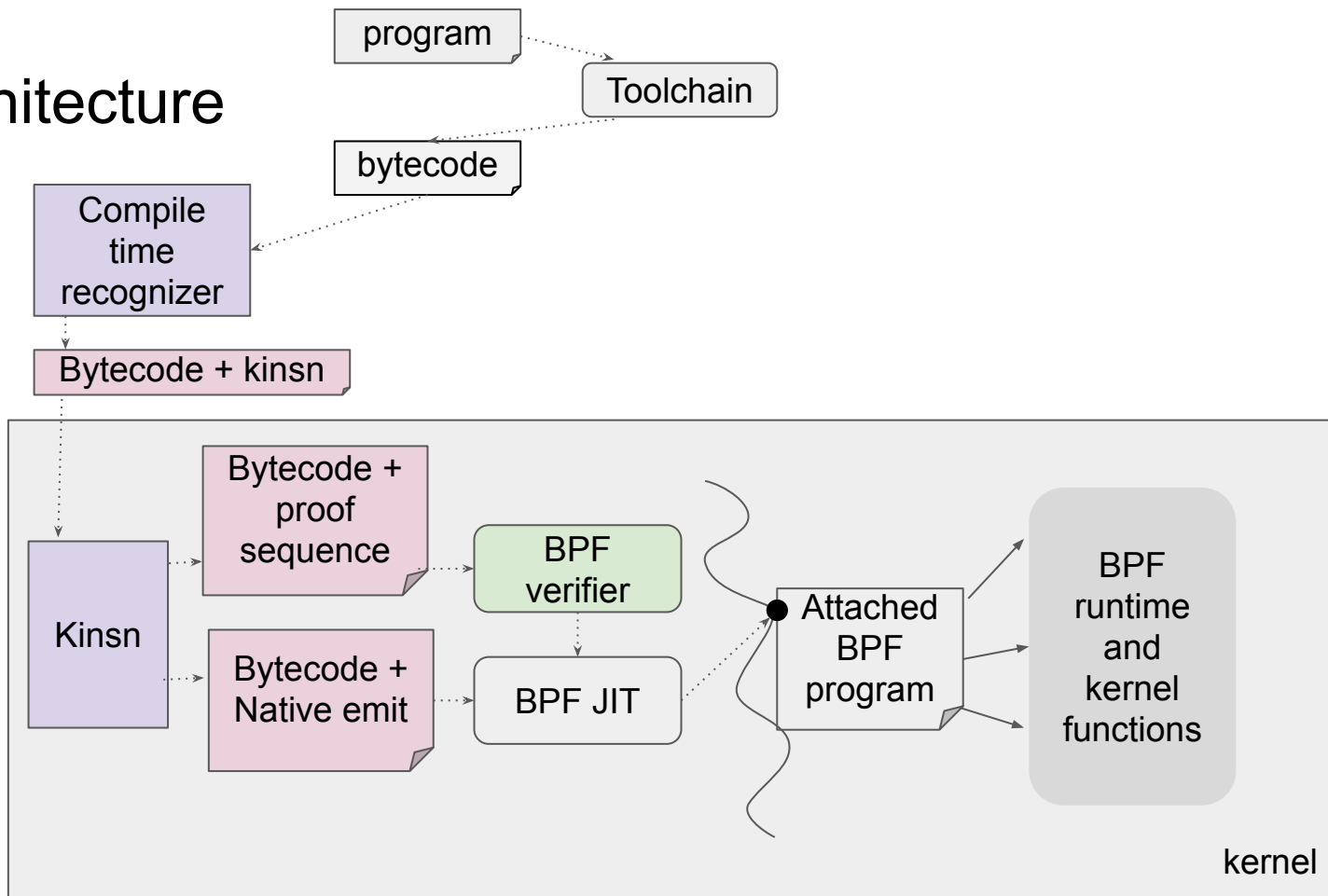
- Introduce new bytecode instructions “kinsn” with
 - Equivalent original bytecode proof sequence
 - Efficient native emit



System Architecture

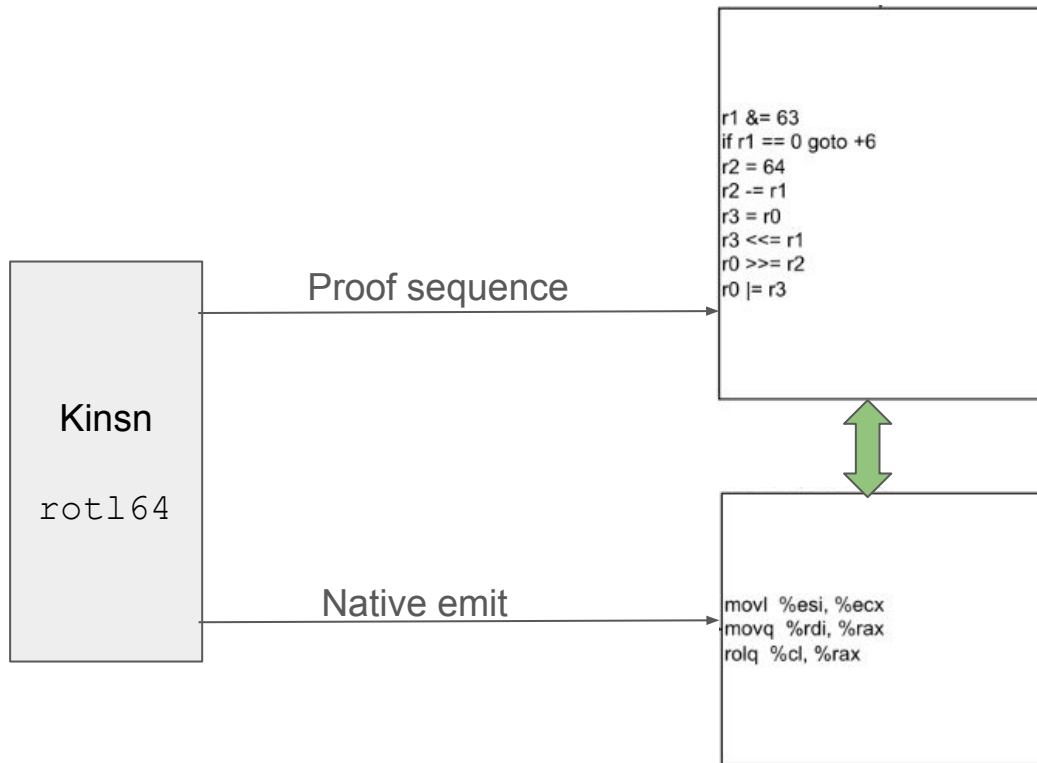


System Architecture



Why is it safe?

- Prove equivalence between forms with respect to BPF-observable architectural state
- Small-step register-transfer semantics
- Lean 4

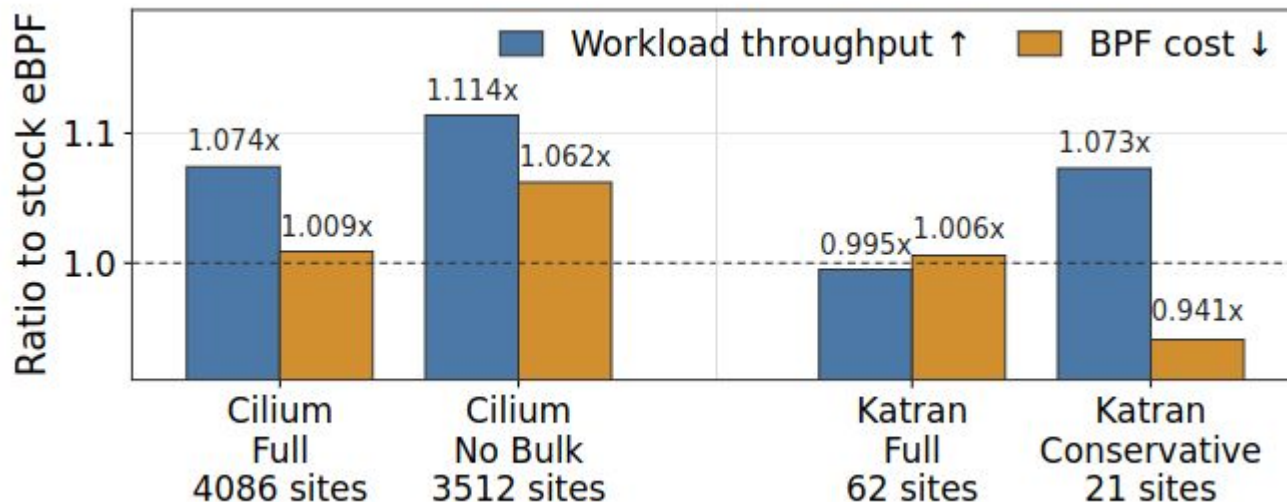


Implementation: 7 kinsns

Operation	Proof sequence	Native emit	
		x86-64	ARM64
Rotate	two shifts + or	ROL	ROR
Conditional select	branch + move	CMP+CMOV	TST+CSEL
Bit extract	shift + and	BEXTR	UBFX
Byte swap	load + byte swap	MOVBE	LDR+REV
Prefetch	no-op (JA +0)	PREFETCHT0	PRFM
Bulk copy	loads + stores	MOV	LDP+STP
LEA	shift + adds	LEA	–

Key result

- Which kinsns to use depends on workload
- Up to 11% performance improvement on Cilium



Lessons from Kinsn

- Safety strategies can have performance implications, even while avoiding pitfalls of other strategies
- Combining multiple strategies and proving equivalence may be a way forward

PART III: Expressiveness

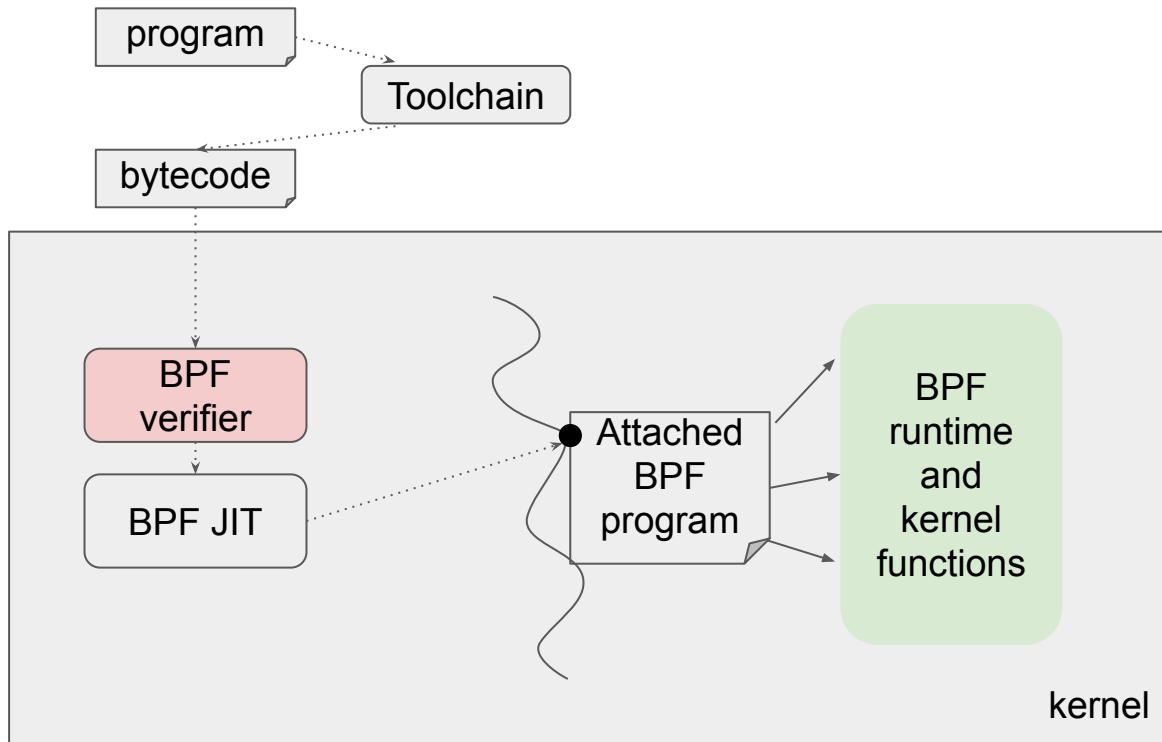
Lead Student:



Tanuj Rao

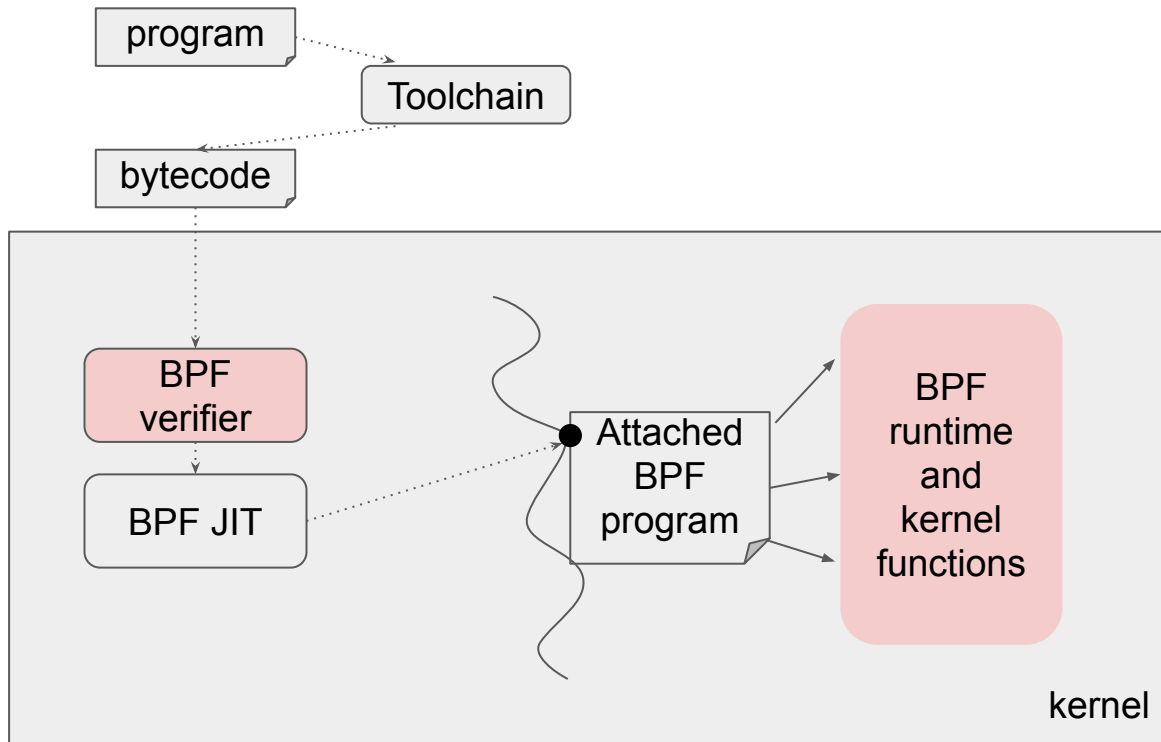
Expressiveness in BPF

- Limited by verifier
 - KFlex, Rex
- Augmented by runtime, kfuncs/helpers
- But runtime has limitations too!



Expressiveness in BPF

- Limited by verifier
 - KFlex, Rex
- Augmented by runtime, kfuncs/helpers
- But runtime has limitations too!



Application acceleration: the frontier of expressiveness

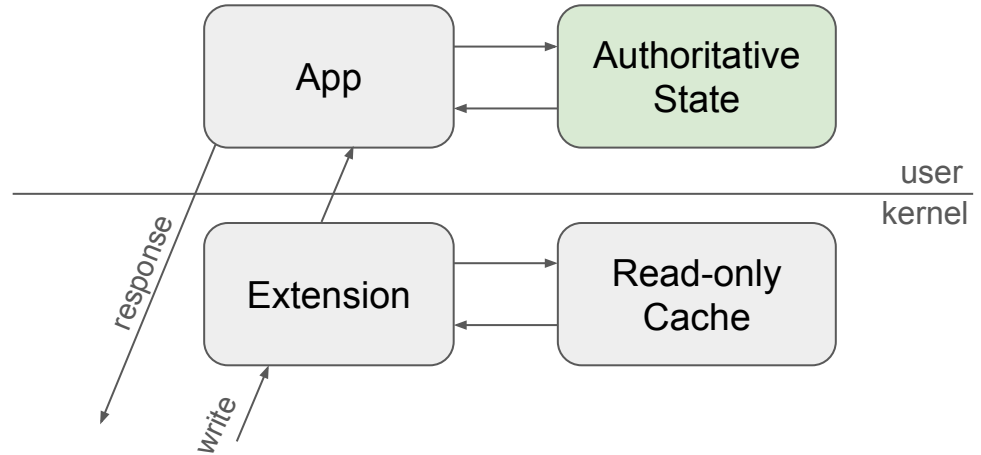
- BPF is no longer only for packet filtering
- Application acceleration via XDP helps avoid slow user/kernel crossings
- Example applications
 - BMC accelerates memcached
 - Electrode accelerates Paxos
 - DINT accelerates locks for distributed transactions
 - BPF-DB accelerates database operations
- But what happens if failures occur?

Userspace to the rescue!!??

- Need to keep authoritative state durable somewhere
- BPF has no support for durable state
- Leads to ***split dataplane*** antipatterns/workarounds:
 - Shadow state
 - Ad hoc boundary protocol
 - Asynchronous sidecar
- These workarounds are for BPF ***runtime*** deficiencies, not the verifier

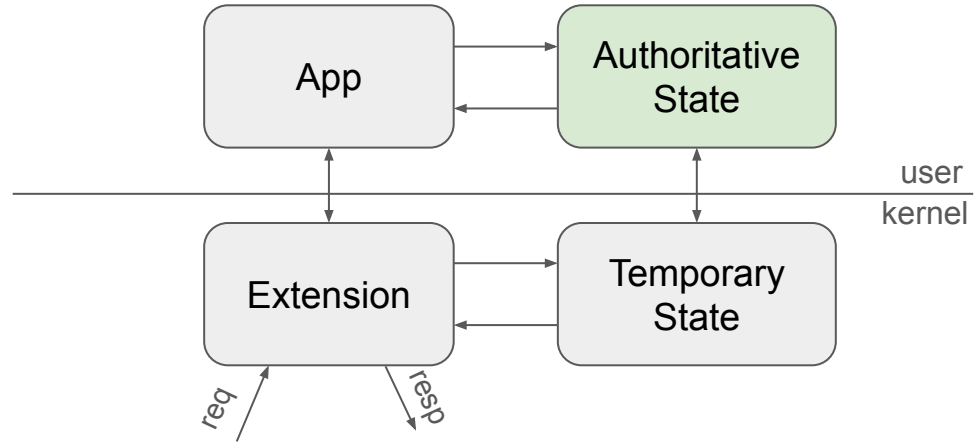
Example split dataplane 1: shadow state

- Extension maintains read-only cache
- Simple, but updates are slow!
- Examples: BMC for GET (not for SET)



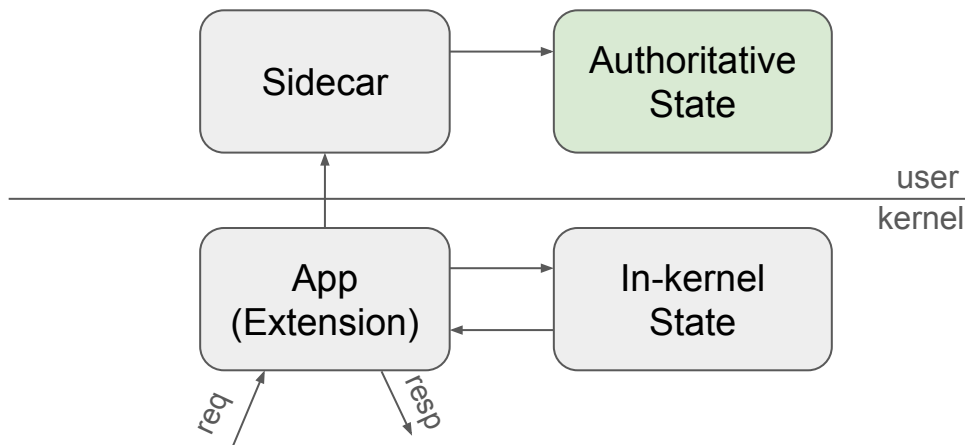
Split dataplane 2: ad-hoc boundary protocols

- Extension acts on most state, but durability handled by userspace
- Complex to detach/reattach extensions, release locks, replay packets, resynch kernel state, etc.
- Examples: Electrode recovery, DINT rare paths



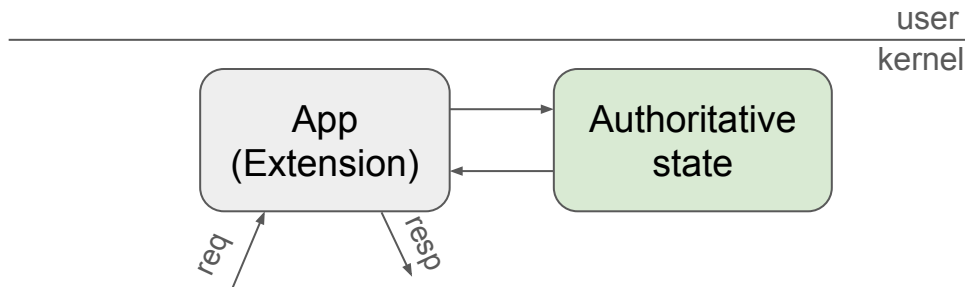
Split dataplane 3: asynchronous sidecar

- Entire application runs in kernel with best effort sidecar updating state
- Reduced guarantees
- Examples: BPF-DB



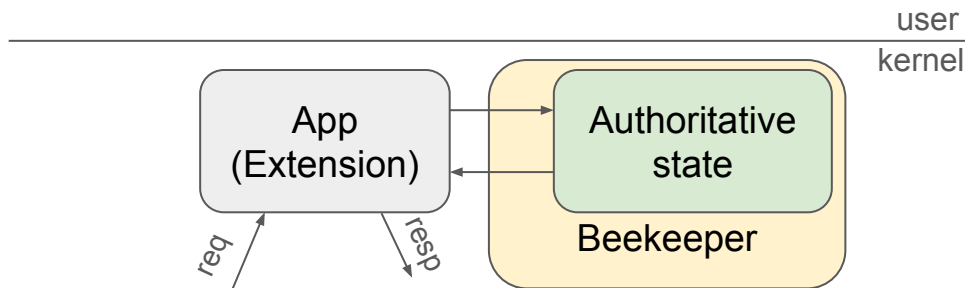
Key idea

- This is not a verifier problem
- Add authoritative, durable state to BPF runtime
 - Q: How to interact with I/O (async) in early network hook?



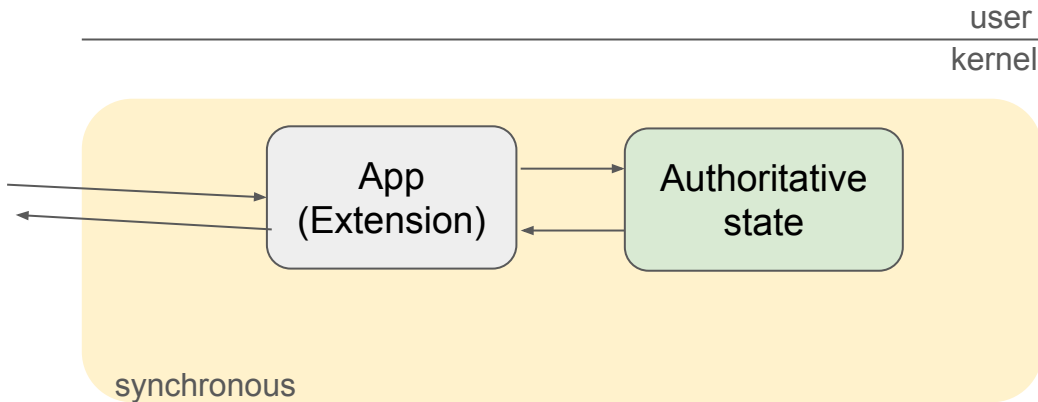
Key idea

- This is not a verifier problem
- Add authoritative, durable state to BPF runtime
 - Q: How to interact with I/O (async) in early network hook?
- Beekeeper
 - Durable KV store
 - 2.94x - 5.47x higher throughput than split dataplane shadow state



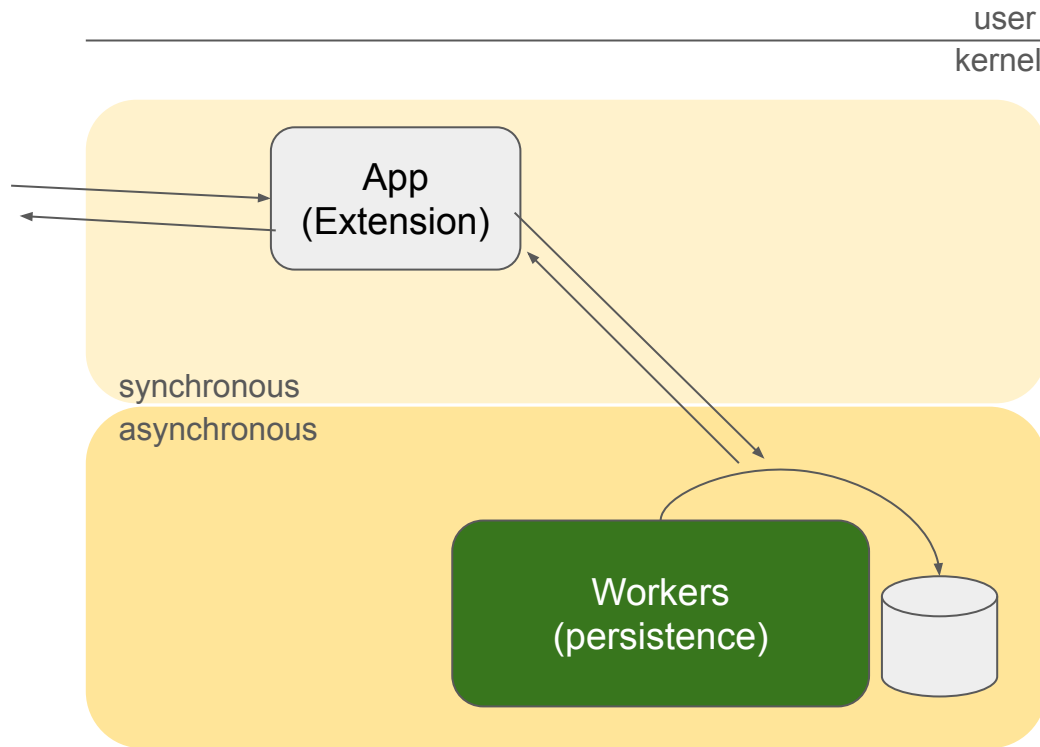
Beekeeper Overview

- Early network hooks (XDP) cannot block!
- Packet lifecycle tied to hook
- Extension-related work is no longer bounded by verifier
- Extension state may become inconsistent with durable state



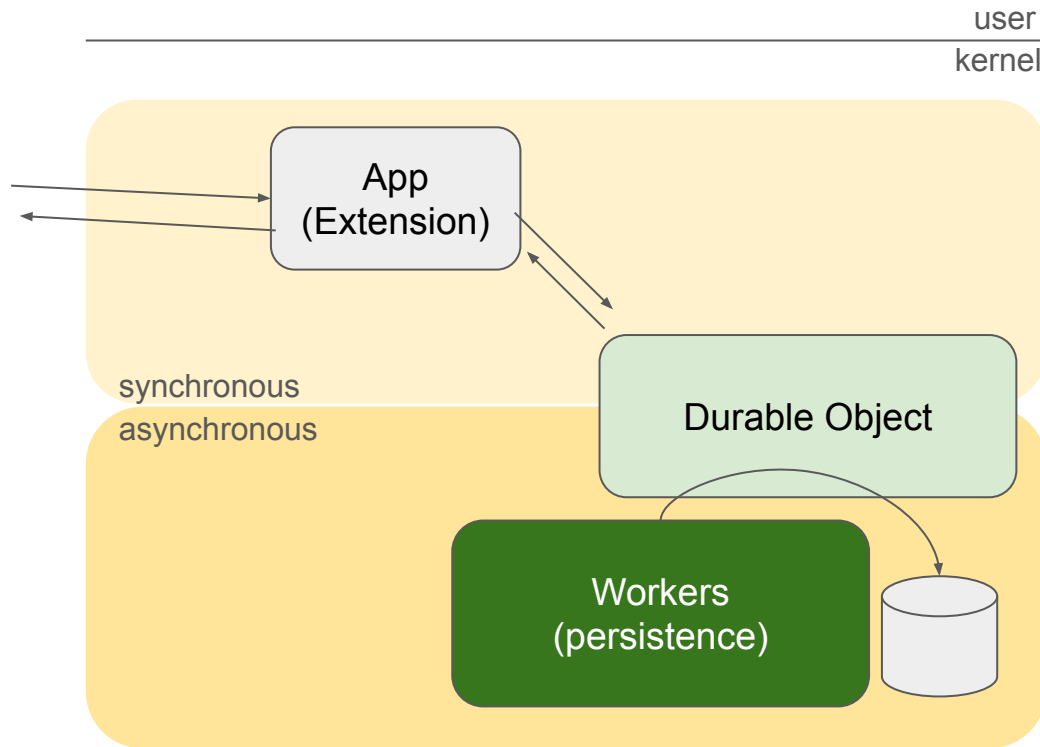
Beekeeper Overview

- Early network hooks (XDP) cannot block!
 - **Defer completion** to asynchronous path
- Packet lifecycle tied to hook
- Extension-related work is no longer bounded by verifier
- Extension state may become inconsistent with durable state



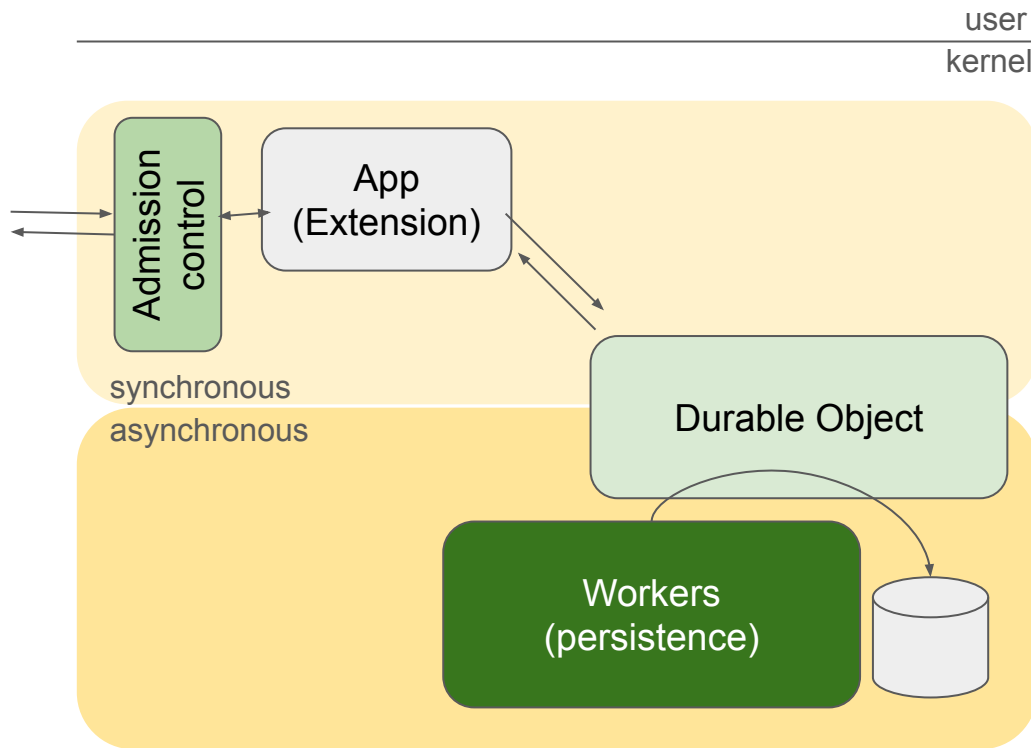
Beekeeper Overview

- Early network hooks (XDP) cannot block!
 - **Defer completion** to asynchronous path
- Packet lifecycle tied to hook
 - Track/extend to asynchronous case via **durable object**
- Extension-related work is no longer bounded by verifier
- Extension state may become inconsistent with durable state



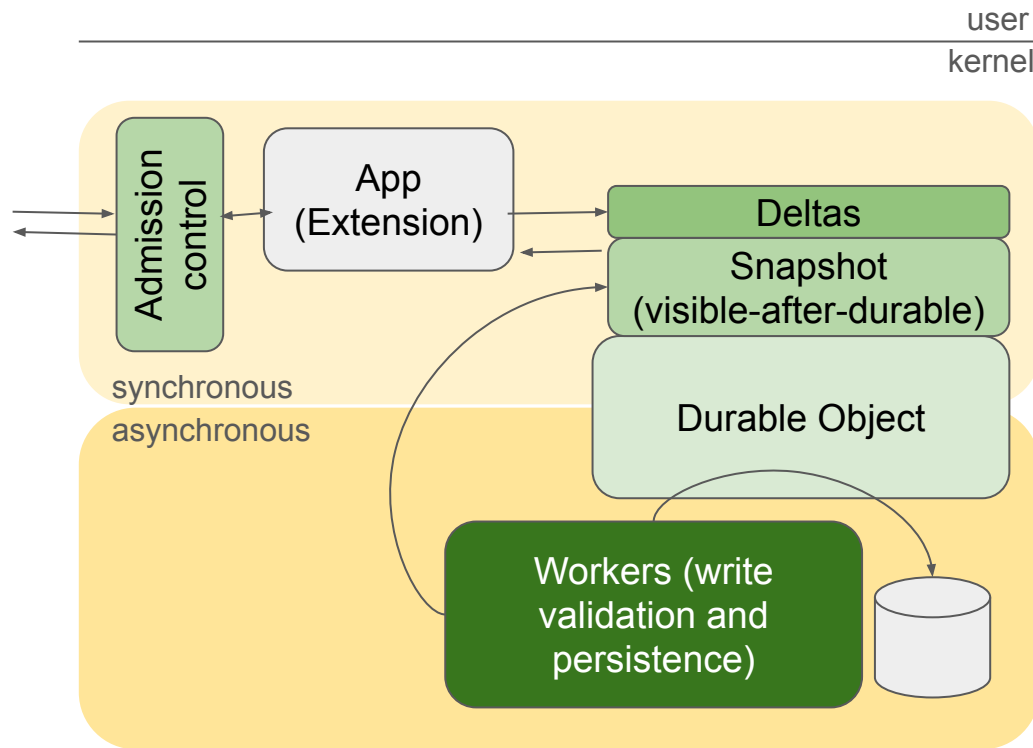
Beekeeper Overview

- Early network hooks (XDP) cannot block!
 - **Defer completion** to asynchronous path
- Packet lifecycle tied to hook
 - Track/extend to asynchronous case via **durable object**
- Extension-related work is no longer bounded by verifier
 - Runtime **admission control** via bounded execution budget
- Extension state may become inconsistent with durable state



Beekeeper Overview

- Early network hooks (XDP) cannot block!
 - **Defer completion** to asynchronous path
- Packet lifecycle tied to hook
 - Track/extend to asynchronous case via **durable object**
- Extension-related work is no longer bounded by verifier
 - Runtime **admission control** via bounded execution budget
- Extension state may become inconsistent with durable state
 - Maintain **visible-after-durable** property through durable object **snapshots**, execution-local **deltas**, and write **validation**



Implementation: BPF maps

- New map types!

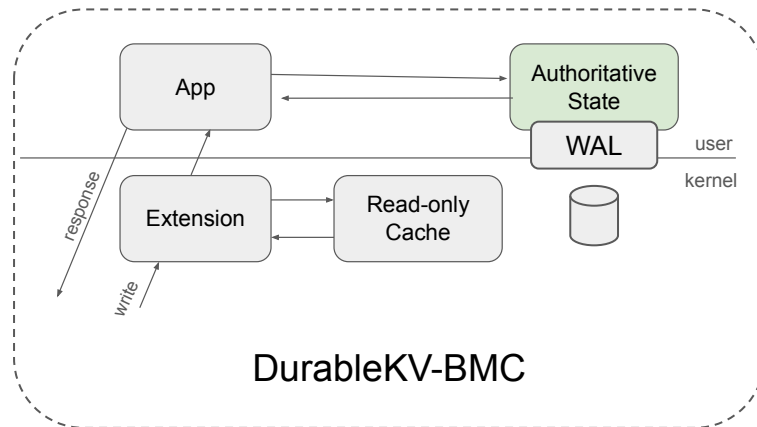
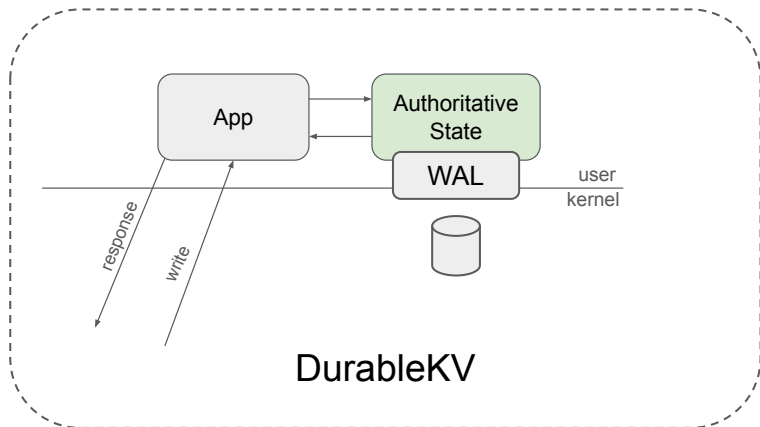
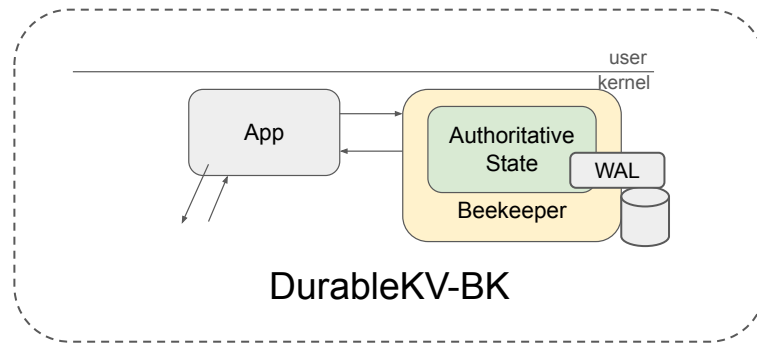
- `BPF_MAP_TYPE_HASH` $\Rightarrow$ `BEEKEEPER_MAP_TYPE_HASH`
- `BPF_MAP_TYPE_ARRAY` $\Rightarrow$ `BEEKEEPER_MAP_TYPE_ARRAY`

- Familiar map interfaces:

- `bpf_map_update_elem` Write to the delta, the update is staged
- `bpf_map_lookup_elem` Check deltas, then snapshot before returning read-only pointer

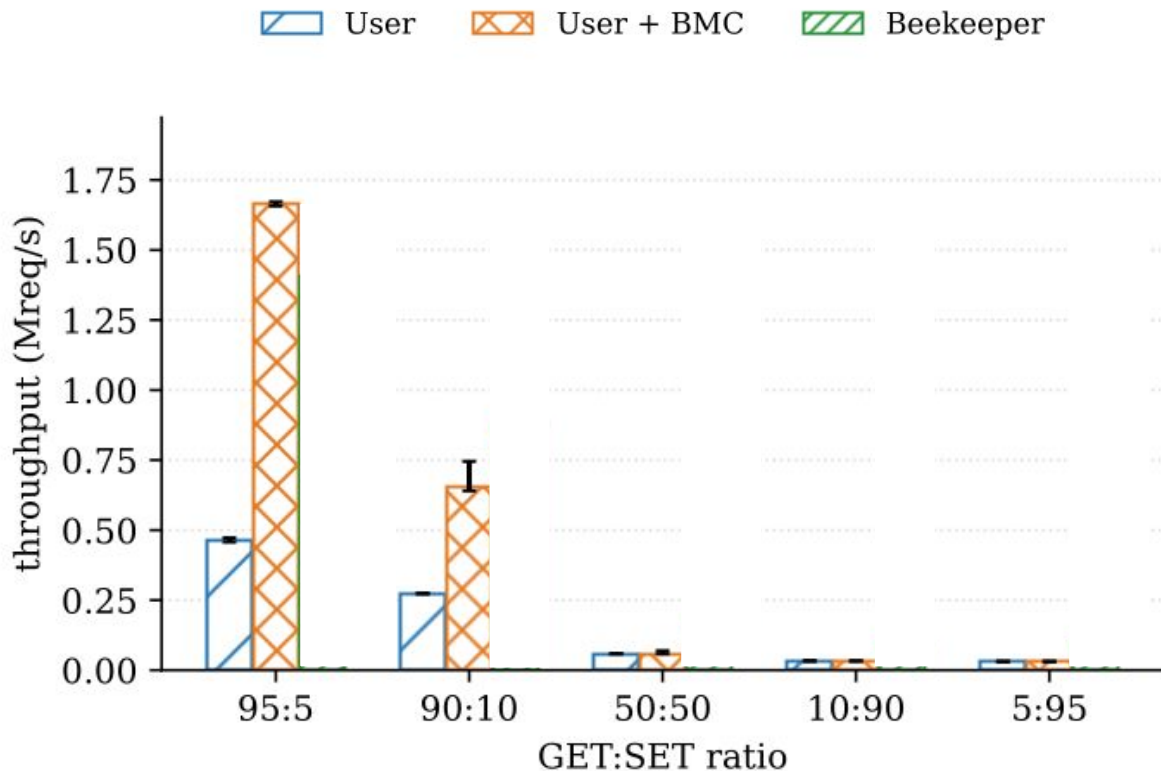
Implementation: DurableKV applications

- 3 crash consistent key value stores
 - Memcached server with write ahead log (WAL) for authoritative state



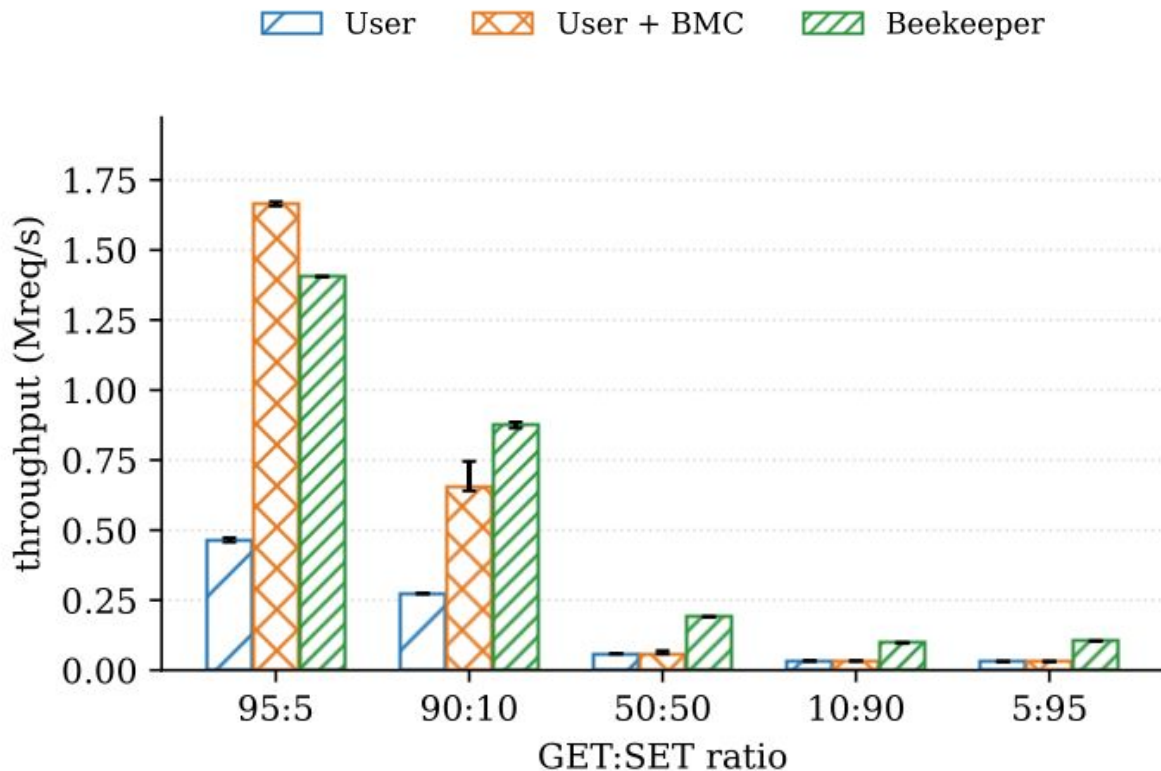
Throughput

- Write mixes limit throughput
- Accelerating reads via split dataplane does not help under writes



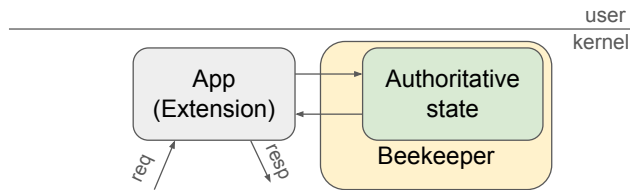
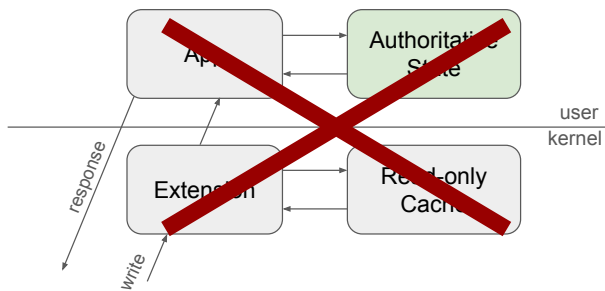
Throughput

- Write mixes limit throughput
- Accelerating reads via split dataplane does not help under writes
- Beekeeper achieves >3x the throughput of DurableKV



Takeaway

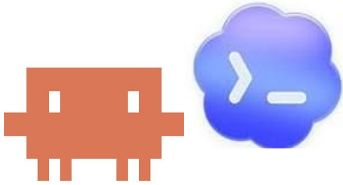
- By enhancing the BPF runtime, data plane no longer must be split for crash consistent stateful applications!
- All possible with current verifier!



Final Food for Thought

- **Safety:**
 - BPF's verifier + safe runtime provides a framework for thinking about safety even in other non-BPF architectures (Rex)
- **Ergonomics:**
 - What do things like the language-verifier gap mean for coding agents?
- **Performance:**
 - Safety strategies can have performance implications, even while avoiding pitfalls of other strategies
 - Native instructions are not all created equal
- **Expressiveness:**
 - Not only verifier, but runtime impacts expressiveness
 - Don't forget safety!

The Era of Coding Agents

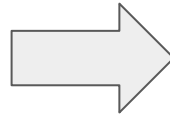


```
226 fflush(stdin);
227 scanf("%s", R.note);
228 if(fwrite(&R, sizeof(R), 1, fp)){
229     gotoxy(5,12);
230     puts("Note is saved successfully");
231     fclose(fp);
232 }
233 else{
234     gotoxy(5,12);
235     SetColor(12);
236     puts("Fail to save!!");
237     ClearColor();
238     gotoxy(5,15);
239     printf("Press any key.....");
240     getch();
241     fclose(fp);
242 }
```

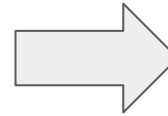
Agent-written

“Software is eating the world.”
– Marc Andreessen

Explosion of
(often specialized)
agent-written code



Humans can't
keep up for
oversight



Increased reliance
on properties of
program toolchains
and execution
environments

BPF
provides a
path forward!

The End

To Verify Or Not to Verify

BPF workshop keynote

September, 2026

Dan Williams

